

Software Testing Exercises with Solutions

1 Black-Box Testing

Exercise 1: Input Validation for Age

Code Snippet

```
#include <iostream>
using namespace std;

string checkEligibility(int age) {
    if (age >= 18 && age <= 60) {
        return "Eligible";
    } else {
        return "Not Eligible";
    }
}
```

Question

Use **Equivalence Class Partitioning (ECP)** and **Boundary Value Analysis (BVA)** to design black-box test cases for the function `checkEligibility`. Identify the valid and invalid partitions, and list the boundary test cases.

Solution

Equivalence Class Partitioning (ECP):

- Valid class: Age between 18 and 60 (inclusive) $\Rightarrow$ e.g., 30
- Invalid class 1: Age $< 18 \Rightarrow$ e.g., 10
- Invalid class 2: Age $> 60 \Rightarrow$ e.g., 70

Boundary Value Analysis (BVA):

- Lower boundary: 18 $\Rightarrow$ Test with 17, 18, 19

- Upper boundary: 60 $\Rightarrow$ Test with 59, 60, 61

Test Cases:

Test Case ID	Input (age)	Reason / Test Technique	Expected Output
TC1	30	Valid ECP	Eligible
TC2	10	Invalid ECP (<18)	Not Eligible
TC3	70	Invalid ECP (>60)	Not Eligible
TC4	17	BVA (just below lower)	Not Eligible
TC5	18	BVA (lower boundary)	Eligible
TC6	19	BVA (just above lower)	Eligible
TC7	59	BVA (just below upper)	Eligible
TC8	60	BVA (upper boundary)	Eligible
TC9	61	BVA (just above upper)	Not Eligible

Exercise 2: Triangle Type Checker

Code Snippet

```
#include <iostream>
using namespace std;

string triangleType(int a, int b, int c) {
    if (a + b > c && a + c > b && b + c > a) {
        if (a == b && b == c) return "Equilateral";
        else if (a == b || b == c || a == c) return "Isosceles";
        else return "Scalene";
    } else {
        return "Not a triangle";
    }
}
```

Question

Using **Equivalence Class Partitioning (ECP)** and **Boundary Value Analysis (BVA)**, derive black-box test cases for the function `triangleType`. Identify valid/invalid partitions and appropriate boundary values.

Solution

Equivalence Class Partitioning (ECP):

- Valid triangle classes:
 - Equilateral: all sides equal $\Rightarrow$ (5,5,5)

- Isosceles: two sides equal $\Rightarrow (5,5,3)$
- Scalene: all sides different $\Rightarrow (4,5,6)$
- Invalid triangle (violates triangle inequality): $(1,2,3)$

Boundary Value Analysis (BVA):

- Edge case: $a + b = c \Rightarrow (2,3,5) \Rightarrow$ invalid
- Just above: $a + b = c + 1 \Rightarrow (3,4,6) \Rightarrow$ valid

Test Cases:

Test Case ID	Input (a,b,c)	Reason / Test Technique	Expected Output
TC1	5,5,5	Valid ECP (Equilateral)	Equilateral
TC2	5,5,3	Valid ECP (Isosceles)	Isosceles
TC3	4,5,6	Valid ECP (Scalene)	Scalene
TC4	1,2,3	Invalid ECP (Inequality fail)	Not a triangle
TC5	2,3,5	BVA ($a + b = c$)	Not a triangle
TC6	3,4,6	BVA ($a + b = c + 1$)	Scalene
TC7	0,5,5	Invalid ECP (zero length)	Not a triangle
TC8	-1,5,5	Invalid ECP (negative length)	Not a triangle

The above solution is based-on well-known triangle classification and inequality conditions.

2 Unit Testing

Exercise 1: Palindrome Checker

Code Snippet

```
#include <string>
using namespace std;

bool isPalindrome(const string& s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        if (s[left] != s[right])
            return false;
        left++;
        right--;
    }
    return true;
}
```

Task

1. Identify and write **unit test cases** for the function `isPalindrome`. 2. Implement a **test suite** using the **Google Test** framework.

Test Case Design

Test Case ID	Input	Description	Expected Output
TC1	"racecar"	Valid palindrome (odd length)	true
TC2	"abba"	Valid palindrome (even length)	true
TC3	"hello"	Not a palindrome	false
TC4	"a"	Single character	true
TC5	""	Empty string	true
TC6	"ab"	Two different characters	false
TC7	"aa"	Two same characters	true

Exercise 2: Discount Calculator

Code Snippet

```
double calculateDiscount(double price, double discountRate) {  
    if (price < 0 || discountRate < 0 || discountRate > 1)  
        return -1; // invalid input  
    return price * (1 - discountRate);  
}
```

Task

1. Write **unit test cases** covering all valid and invalid inputs. 2. Implement the corresponding **unit test suite** using Google Test.

Test Case Design

Test Case ID	Price	Discount Rate	Description	Expected Output
TC1	100	0.2	Valid input (20% discount)	80
TC2	100	0.0	No discount	100
TC3	100	1.0	Full discount	0
TC4	-10	0.2	Negative price	-1
TC5	100	-0.1	Negative discount rate	-1
TC6	100	1.1	Discount greater than 100%	-1
TC7	0	0.5	Zero price	0

3 Integration Testing

Exercise 1: Banking System – Transfer Funds

Code Snippet

```
class Account {
private:
    std::string owner;
    double balance;
public:
    Account(const std::string& owner, double balance)
        : owner(owner), balance(balance) {}

    bool debit(double amount) {
        if (amount < 0 || amount > balance)
            return false;
        balance -= amount;
        return true;
    }
    void credit(double amount) {
        if (amount >= 0)
            balance += amount;
    }
    double getBalance() const {
        return balance;
    }
};

bool transfer(Account& from, Account& to, double amount) {
    if (from.debit(amount)) {
        to.credit(amount);
        return true;
    }
    return false;
}
```

Integration Test Cases

Test Case ID	From Bal.	To Bal.	Amount	Description	Expected Outcome
TC1	1000	500	200	Valid transfer	true, balances updated
TC2	100	300	150	Insufficient funds	false, no change
TC3	500	800	0	Zero amount	true, no change
TC4	500	400	-50	Negative transfer	false, no change

Exercise 2: Library Management – Book Issuing

Code Snippet

```
#include <string>
#include <unordered_map>

class Book {
public:
    std::string title; bool isIssued;

    Book(const std::string& title) : title(title), isIssued(false)
    {}

    bool issue() {
        if (isIssued)
            return false;
        isIssued = true;
        return true;
    }
    void returnBook() {
        isIssued = false;
    }
};

class Library {
private:
    std::unordered_map<std::string, Book> books;
public:
    void addBook(const std::string& title) {
        books[title] = Book(title);
    }

    bool issueBook(const std::string& title) {
        if (books.find(title) == books.end()) return false;
        return books[title].issue();
    }
    void returnBook(const std::string& title) {
        if (books.find(title) != books.end())
            books[title].returnBook();
    }
    bool isIssued(const std::string& title) const {
        auto it = books.find(title);
        return it != books.end() && it->second.isIssued;
    }
};
```

Integration Test Cases

Test Case ID	Book Title	Scenario	Expected Outcome
TC1	C++ Primer	Issue an available book	true, isIssued = true
TC2	C++ Primer	Issue an already issued book	false, isIssued = true
TC3	C++ Primer	Return a book	isIssued = false
TC4	Unknown Book	Try issuing non-existent book	false

4 Integration and White-Box Testing

Exercise 3: White Box Testing – Grading System

Code Snippet

```
char grade(int marks) {  
    if (marks >= 90) return 'A';  
    else if (marks >= 80) return 'B';  
    else if (marks >= 70) return 'C';  
    else if (marks >= 60) return 'D';  
    else return 'F';  
}
```

White Box Test Cases (Statement Coverage)

Test Case ID	Input	Expected Output	Coverage (%)
TC1	95	'A'	20%
TC2	85	'B'	40%
TC3	75	'C'	60%
TC4	65	'D'	80%
TC5	50	'F'	100%

Total Coverage: 100%

Exercise 4: White Box Testing – Absolute Difference

Code Snippet

```
int absoluteDiff(int a, int b) {  
    if (a > b)  
        return a - b;  
    else if (b > a)  
        return b - a;  
    else  
        return 0;  
}
```

White Box Test Cases (Statement Coverage)

Test Case ID	Input (a,b)	Expected Output	Coverage (%)	Description
TC1	(10, 5)	5	33%	First path: a > b
TC2	(5, 10)	5	66%	Second path: b > a
TC3	(7, 7)	0	100%	Third path: a == b

Total Coverage: 100%

Exercise 5: White Box Testing – Branch Coverage (Leap Year Check)

Code Snippet

```
bool isLeapYear(int year) {  
    if (year % 4 != 0)  
        return false;  
    else if (year % 100 != 0)  
        return true;  
    else if (year % 400 == 0)  
        return true;  
    else  
        return false;  
}
```

Branch Coverage Test Cases

Test Case ID	Input	Expected Output	Coverage	Description
TC1	2023	false	25%	First condition false
TC2	2024	true	50%	First true, second true
TC3	2100	false	75%	First true, second false, third false
TC4	2000	true	100%	All branches explored

Total Branch Coverage: 100%

Exercise 6: White Box Testing – Branch Coverage (Login System)

Code Snippet

```
bool authenticate(const std::string& username, const std::string& password) {  
    if (username.empty() || password.empty())  
        return false;  
    if (username == "admin" && password == "admin123")  
        return true;  
    return false;  
}
```

Branch Coverage Test Cases

Test Case ID	Username	Password	Expected Output	Coverage	Description
TC1	""	"admin123"	false	25%	Username empty
TC2	"admin"	""	false	50%	Password empty
TC3	"admin"	"admin123"	true	75%	Correct credentials
TC4	"user"	"123"	false	100%	Invalid credentials

Total Branch Coverage: 100%

Exercise 7: White Box Testing – Condition Coverage (Voting Eligibility)

Code Snippet

```
bool canVote(bool isCitizen, int age) {  
    if (isCitizen && age >= 18)  
        return true;  
    return false;  
}
```

Condition Coverage Test Cases

Test Case ID	isCitizen	age	Expected Output	Coverage	Condition Evaluation
TC1	true	20	true	50%	isCitizen=true, age >=18=true
TC2	false	20	false	75%	isCitizen=false, age >=18=true
TC3	true	16	false	100%	isCitizen=true, age <=18=false

Total Condition Coverage: 100%

Exercise 8: White Box Testing – Condition Coverage (Number Range Check)

Code Snippet

```
bool isInRange(int x) {  
    if ((x >= 10) || (x == 5))  
        return true;  
    return false;  
}
```

Condition Coverage Test Cases

Test Case ID	Input	Expected Output	Coverage	Condition Evaluation
TC1	12	true	50%	x >= 10=true, x == 5=don't care
TC2	5	true	75%	x <= 10=false, x == 5=true
TC3	2	false	100%	x <= 10=false, x == 5=false

Total Condition Coverage: 100%